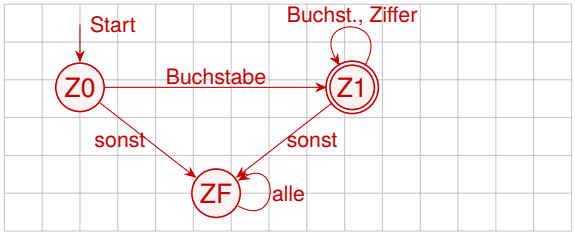
1.6 Endliche Automaten programmieren

In 1.5 stand der Automat in einer **Übergangstabelle**, also in Daten. Hier steckt er direkt im **Programmcode**. Beispiel: Ein Compiler muss Namen (Bezeichner) wie `zahl2` erkennen, vereinfacht: `Bezeichner = Buchstabe {Buchstabe | Ziffer}.`

Aufgabe 1 *Code lesen: der Bezeichner-Automat*

```
1  public void zustandWechseln(char zeichen) {
2     switch (zustand) {
3        case 0:
4           if (Character.isLetter(zeichen)) {
5              zustand = 1;
6           } else {
7              zustand = ZF;
8           }
9           break;
10       case 1:
11          if (Character.isLetterOrDigit(zeichen)) {
12             zustand = 1;
13          } else {
14             zustand = ZF;
15          }
16          break;
17       default:
18          zustand = ZF;
19    }
20 }
21 public boolean istEndzustand() {
22    return zustand == 1;
23 }
```

a) Zeichne das Zustandsübergangsdiagramm, das im Code steckt, *mit* Fangzustand ZF.



b) Zustandsfolge und Ergebnis:

`zahl2`	Z0 Z1 Z1 Z1 Z1 Z1	ja
`2zahl`	Z0 ZF ZF ZF ZF ZF	nein
ε	Z0	nein

c) In welcher Zeile steht der Pfeil …

Z0 → Z1	Z1 → Z1	Z1 → ZF	ZF → ZF
5	12	14	18

Einen Automaten als Code programmieren

Zustand: Das Attribut `zustand` merkt sich, wo der Automat gerade steht (0 für Z0, 1 für Z1, …).

Übergänge: `zustandWechseln(zeichen)` unterscheidet zuerst nach dem aktuellen Zustand (`switch`: je Zustand ein `case`), dann nach dem gelesenen Zeichen (`if`). Jeder Pfeil wird genau eine Anweisung `zustand = …;`

Kein passender Pfeil: Der `else`-Zweig führt in den Fangzustand ZF, der `default`-Zweig sorgt dafür, dass ZF nie verlassen wird.

Endzustände: `istEndzustand()` vergleicht `zustand` mit allen Endzuständen.

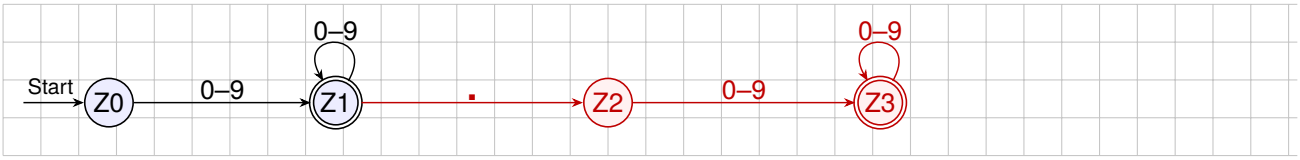
Gegeben: abstrakte Oberklasse `DEA`

```
public boolean pruefe(String wort) {
   zustand = 0;                                  // ①
   for (int i = 0; i < wort.length(); i++) {     // ③
      zustandWechseln(wort.charAt(i));           // ②
   }
   return istEndzustand();                       // ④
}
```

`pruefe` ist für alle Automaten gleich und steht deshalb in der Oberklasse `DEA`. Jede Unterklasse legt nur die Übergänge und die Endzustände fest.

Aufgabe 2 *Verändern: Kommazahlen* (Projekt: TODO 2)

`Kommazahl` erkennt bisher nur ganze Zahlen wie `42`. Künftig sollen auch `3.14` und `0.5` akzeptiert werden, `3.` und `.5` nicht. **a)** Ergänze das Diagramm um Z2 und Z3 (fehlende Pfeile führen in ZF).



b) Ergänze im `case 1` den Pfeil Z1 → Z2 (TODO 2a). Die Fälle für Z2 und Z3 programmierst du im Projekt (TODO 2b).

```
if (Character.isDigit(zeichen)) {
   zustand = 1;
} else if (zeichen == '.') {
     zustand = 2;
} else {
   zustand = ZF;
}
```

c) TODO 2c: `istEndzustand()` liefert `return` `zustand == 1 || zustand == 3` ;

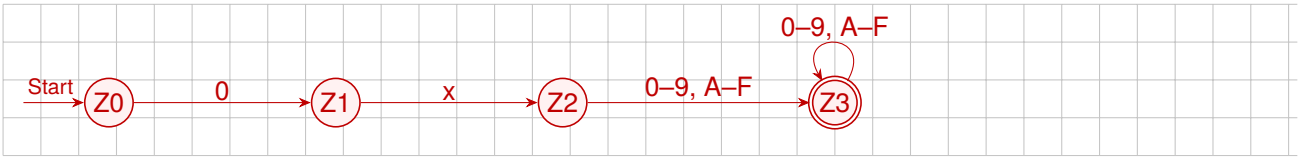
d) `3.` landet nie in ZF und wird trotzdem abgelehnt. Warum? Es endet in Z2, kein Endzustand (Regel ④).

Aufgabe 3 *Selbst bauen: Hexadezimalzahlen* (Projekt: TODO 3)

In Java schreibt man Hexzahlen mit `0x` davor, z. B. `0x1F` (= 31) oder `0xFF` (= 255):

```
Hexzahl = "0" "x" HexZiffer {HexZiffer}.
HexZiffer = "0" | … | "9" | "A" | … | "F".
```

a) Zeichne den deterministischen Automaten (ohne ZF). **b)** Programmiere ihn im Projekt (`Hexzahl`).



c) Anzahl der `case`-Zweige (ohne `default`): 4 `istEndzustand()`: `return` `zustand == 3;`

Aufgabe 4 *Vergleichen: Tabelle (Projekt 1.5) oder Code (Projekt 1.6)?*

	Übergangstabelle (1.5)	**Code (1.6)**
Wo stehen die Pfeile?	in Daten: im Feld `tabelle`, gefüllt durch Aufrufe im Hauptprogramm	im Programmtext: in `zustandWechseln` einer Unterklasse
Was tun für einen neuen Automaten?	nur neue Aufrufe (`uebergangHinzufuegen` …); die Klasse bleibt gleich	eine neue Unterklasse von `DEA` schreiben
Ein Pfeil für alle Buchstaben?	je Zeichen eine Spalte: über 50 Spalten nur für Buchstaben	eine einzige Bedingung: `Character.isLetter(zeichen)`